
TD7: Ingeniería de Datos

Funcionamiento de S3

Parte 1 – Conceptos básicos y consola

- ¿Qué es S3?
- ¿Qué es un bucket?
- Crear un bucket desde la consola
- Subir archivos manualmente

S3 (Simple Storage Service) es el servicio de almacenamiento de objetos de AWS: un disco rígido en la nube, infinito y accesible desde cualquier lugar.

Conceptos claves:

Bucket — el contenedor principal. Es como una carpeta raíz donde se guarda todo. **El nombre tiene que ser único a nivel mundial en AWS.**

¿Para qué se usa S3?

Guardar imágenes y videos de aplicaciones

Almacenar backups y logs

Hospedar sitios web estáticos

Data lake para análisis (que es lo que vamos a hacer con **Athena**)

Conceptos claves:

Objeto — cualquier archivo que se suba: imágenes, CSVs, JSONs, backups, logs, etc.

Cada objeto tiene:

Una **clave** (el nombre/ruta del archivo)

El **contenido** (los datos)

Metadata (información adicional)

Región — el bucket vive en una región específica de AWS (como sa-east-1). Podría convenir que esté en la misma región que otros servicios para mayor velocidad y menor costo.

Crear el bucket desde la consola

1. Buscar "S3"

2. "Create bucket"

- **Bucket name:** Tiene que ser único a nivel mundial.
- **AWS Region:** sa-east-1
- **Object Ownership: ACLs disabled** (recomendado).

Significa que estás usando el modelo recomendado por AWS hoy: **control de acceso solo con políticas**. El bucket siempre es dueño de todo

Crear el bucket desde la consola

- **Block Public Access: Block all public access** (por ahora — lo cambiamos después si necesitamos acceso público).

Seguridad por defecto (principio de mínimo privilegio): nada es público, todo requiere autenticación. Se desactiva sólo cuando realmente se necesita acceso público.

- **Versioning: Disable** (por ahora).

Cuando activás versioning, cada vez que se sube un archivo con el mismo nombre, **no se reemplaza**, se guarda una **nueva versión**. Y cada versión ocupa espacio y se factura.

- **Encryption:** Dejarlo por defecto.
- **"Create bucket"**

Subir un archivo manualmente desde la consola

3. Primero necesitamos un archivo de prueba.

4. **desde la consola de S3:**

- Entrar al bucket
- **Click en "Upload"**
- Seleccionar el archivo
- Click en **"Upload"**

Explorando el archivo

Desde la consola, click en <nombre del archivo>; ver la información que muestra AWS:

- **URL del objeto** — Cada objeto en S3 tiene una URL única con este formato: <https://nombre-bucket.s3.region.amazonaws.com/nombre-archivo>.

Si se intenta abrir esa URL directamente en el navegador va a dar un error de **Access Denied** — porque el bucket está bloqueado para acceso público.

- **Tamaño**
- **Última modificación**
- **Clase de almacenamiento** — Standard por defecto
- **click en “Abrir”** — se descarga

Parte 2 – AWS CLI con S3

El comando base para S3 en AWS CLI es **aws s3**

Primero hay que agregarle la policy correspondiente al usuario correspondiente que todavía no tiene permisos para S3.

Desde la consola de AWS con usuario root:

- IAM → Usuarios de IAM → <nombre usuario>
- Pestaña "Permissions"
- "Add permissions" → "Attach policies directly"
 - Buscar **AmazonS3FullAccess**
- "Next" → "Add permissions"

Ejecutar **aws s3 ls**

Probando comandos

- Listar el contenido del bucket
`aws s3 ls s3://clase-utdt`
- Subir un archivo desde CLI:
`aws s3 cp datos2.txt s3://clase-utdt/datos2.txt`
- Bajar un archivo desde S3 a tu máquina:
`aws s3 cp s3://clase-utdt/datos20260330.csv ./datos-descargado.csv`
- Listar recursivamente (si hay carpetas):
`aws s3 ls s3://clase-utdt --recursive`
- Borrar un archivo:
`aws s3 rm s3://clase-utdt /datotestutdts2.csv`
- Sincronizar una carpeta local con el bucket:
`aws s3 sync ./mi-carpeta s3://test-clase-utdt`

El comando sync es muy poderoso — solo sube los archivos que cambiaron, igual que rsync pero para S3. Muy usado para backups y deploys.

Resumen de los comandos:

aws s3 ls

Listar buckets

aws s3 ls s3://bucket

Listar contenido de un bucket

aws s3 cp archivo s3://bucket

Subir archivo

aws s3 cp s3://bucket/archivo .

Bajar archivo

aws s3 rm s3://bucket/archivo

Borrar archivo

aws s3 sync ./carpeta s3://bucket

Sincronizar carpeta

Parte 3 – S3 + Athena

- Subir un CSV a S3
- Crear una tabla en Athena apuntando al bucket
- Hacer consultas SQL sobre los datos

¿Qué es Athena?

- Permite hacer consultas SQL directamente sobre archivos que están en S3 — sin base de datos, sin cargar datos, sin infraestructura. Solo SQL sobre archivos.
- **El flujo:**
datos.csv en S3 → Athena define la estructura → consulta con SQL

Arrancamos desde la consola:

- En la barra de búsqueda "**Athena**"
- La URI debe ser algo así: <https://sa-east-1.console.aws.amazon.com/athena/home?region=sa-east-1#/query-editor> (debe terminar en "query-editor")
- **Antes de consultar, Athena necesita guardar los resultados en algún lugar de S3.** Se puede usar el mismo bucket:
s3://clase-utdt/resultados-athena/



Realización de consultas en Amazon SageMaker Unified Studio

Consulta ↗



Utilice el rol de IAM para el análisis y la creación de los recursos de AWS en un único entorno de desarrollo de datos e IA.

Editor

Consultas recientes

Consultas guardadas

Configuración de consultas

☒ Grupo de trabajo ☐ Entorno de DataZone

primary ▼

ⓘ Antes de ejecutar la primera consulta, debe configurar una ubicación del resultado de la consulta en Amazon S3.

Editar ajustes

ⓘ Athena ahora admite sugerencias de código de escritura anticipada para acelerar el desarrollo de consultas SQL

Las sugerencias de escritura anticipada están activadas de forma predeterminada. Puede cambiar esta configuración en las preferencias del editor de consultas.

Editar preferencias



Configurar dónde Athena guarda los resultados

1. En Configuración de consultas /
Cifrado de resultados de consultas
2. "Administrar"
3. Poner una ruta con este formato:
`s3://clase-utdt/resultados-athena/`

Crear la tabla que apunta al csv creado en S3

- En el Query Editor, arriba del editor hay un selector que dice "**Database**" — verificar que esté seleccionado "**default**"
- En el Query Editor pegar esta consulta SQL:

```
CREATE EXTERNAL TABLE IF NOT EXISTS datos_clase (  
    nombre STRING,  
    edad INT,  
    ciudad STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
LINES TERMINATED BY '\n'  
LOCATION 's3://clase-utdt/'  
TBLPROPERTIES ('skip.header.line.count'='1');
```
- **Ejecutar**

Consultas SQL

¿Qué hace esto? Le dice a Athena: *“los archivos que están en ese bucket tienen estas tres columnas; tratarlas como una tabla llamada datos_clase”*

```
SELECT * FROM datos_clase;
```

Para destacar:

- No hay base de datos corriendo
- No se cargaron los datos en ningún lado
- Es sólo SQL sobre un archivo CSV en S3
- Se paga sólo por los datos que se escanean; con este archivo el costo es cero.

Consultas SQL

Athena no cobra por tener servidores ni bases de datos. Cobra por la cantidad de datos que necesita leer de S3 para ejecutar la consulta.

Por ejemplo:

- Si se tiene un CSV de **1 KB**, Athena escanea 1 KB → costo prácticamente nulo.
- Si se tiene un CSV de **10 GB** y se hace la consulta:

```
SELECT * FROM .....;
```
- Athena probablemente tenga que leer los 10 GB completos.
- Si se tiene los mismos datos en formato Parquet particionado y la consulta necesita sólo una parte, quizás Athena lea apenas 100 MB.

Parte 4 – S3 + RDS

- ¿Qué es RDS?
- Importar datos desde S3 a una base de datos RDS

RDS (Relational Database Service)

Es el servicio de bases de datos relacionales administrado de AWS. Soporta varios motores:

- PostgreSQL
- MySQL
- MariaDB
- Oracle
- SQL Server

La diferencia con Athena:

Athena

- Consultas sobre archivos
- S3
- Análisis, reportes
- Por consulta

RDS

- Base de datos real
- En la base de datos
- Aplicaciones en producción
- Por hora de instancia

Los datos viven

Usado para

¿Costo?

El flujo:

datos.csv en S3 → importar a RDS → consultar con SQL

Aviso importante antes de arrancar: RDS no está en el free tier de forma indefinida. Se tienen una cantidad de horas gratis por mes con instancias db.t3.micro. Después de eso tiene costo.

Creación de una instancia de RDS

Desde la consola de AWS:

- En la barra de búsqueda: **"RDS"**
- Click en **"Create database" (Configuración completa)**
- **Creación sencilla**
- **DB instance identifier:** el nombre de la base de datos.
- **Master username:** postgres (viene por defecto, dejarlo así)
- **Master password:** necesaria después para conectarse.

Si todo se ve bien, click en **"Create database"**.

- **La instancia tarda unos 5-10 minutos en crearse** —estado "Creating" y después "Available".

Aurora and RDS

Panel

[Bases de datos](#)

Información sobre rendimiento

Instantáneas de

Exportaciones en Amazon S3

Copias de seguridad automatizadas

Instancias reservadas

Proxies

✓ Se ha creado correctamente la base de datos [database-20260330](#)

[Ver detalles de conexión](#)



Puede utilizar la configuración de [database-20260330](#) para simplificar la configuración de **complementos de base de datos sugeridos** mientras terminamos de crear su base de datos.

Bases de datos (1)

☒ Recursos del grupo



[Modificar](#)

[Acciones ▼](#)

[Crear base de datos ▼](#)

< 1 >



 Identificador de base de datos	▲	Estado ▼	Rol ▼	Motor ▼	Orden de implemen
 database-20260330		 Config...	Instancia	PostgreSQL	SECOND

Identificador de base de datos

Cuando se ve:

“Identificador de base de datos: database-1”

eso en **Amazon RDS** significa:

- el nombre de la *instancia* (el servidor)
- **NO** es necesariamente una base de datos interna de PostgreSQL.

Cómo se organiza esto (importante)

En **PostgreSQL** se tiene:

- **Instancia (RDS)** → database-1
- **Bases de datos dentro** → postgres, mydb, etc.

Cómo conectarse con psql

Usar el endpoint de la instancia **PERO** el nombre de una base real.

Conectividad y seguridad → Punto de enlace

Probar algo como esto:

```
psql -h database-1.cnsmmismmkjk.sa-east-1.rds.amazonaws.com -p 5432 -U  
postgres -d postgres
```

En resumen

- database-1 = servidor (RDS)
- postgres = base real (default)
- psql siempre necesita una base existente

Pasos:

- 1. Conectarnos a la base de datos** desde la terminal usando `psql` (el cliente de PostgreSQL)
- 2. Crear una tabla** en la base de datos
- 3. Importar los datos** del CSV en S3
- 4. Consultar** con SQL

Para conectarnos vamos a necesitar:

- El **endpoint** de la instancia (la URL que da AWS)
- El **puerto** (5432 por defecto en PostgreSQL)
- El **usuario** (`postgres`)
- La **contraseña**

Dentro de PostgreSQL

- **Paso 1 — Creamos la base de datos y la tabla:**

```
CREATE DATABASE clase;
```

Después conectarse a la base:

```
\c clase
```

Y crear la tabla:

```
CREATE TABLE personas (  
    nombre VARCHAR(100),  
    edad INT,  
    ciudad VARCHAR(100)  
);
```

```
\dt
```

```
postgres=> CREATE DATABASE clase;
CREATE DATABASE
postgres=> \c clase
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "clase" as user "postgres".
clase=> CREATE TABLE personas (
clase(>     nombre VARCHAR(100),
clase(>     edad INT,
clase(>     ciudad VARCHAR(100)
clase(> );
CREATE TABLE
clase=> \dt
          List of relations
 Schema |   Name   | Type  | Owner
-----+-----+-----+-----
 public | personas | table | postgres
(1 row)

clase=>
```

Importar los datos del csv en S3

En PostgreSQL hay varias formas de importar datos desde S3. La más directa es usando la extensión **aws_s3** que tiene RDS.

- **Paso 2 — Habilitamos la extensión:**

```
CREATE EXTENSION aws_s3 CASCADE;
```

Tiene que devolver

```
NOTICE: installing required extension "aws_commons"  
CREATE EXTENSION
```

La extensión se instaló correctamente junto con aws_commons que es una dependencia.

RDS necesita permisos para acceder a S3

Hay que crear un rol IAM que le permita a RDS leer desde S3.

Desde la consola de AWS:

- **IAM → Roles → "Create role"**
- **Trusted entity:** elegir **"AWS service"**
- **Use case:** buscar y elegir **"RDS"** → elegir **"RDS - Add Role to Database"**
- **"Next"**
- Buscar y tildar **AmazonS3ReadOnlyAccess**
- **"Next"**
- **Role name:** rds-s3-import-roleUTDT
- **"Create role"**

RDS necesita permisos para acceder a S3

El rol tiene que estar **asociado a la instancia RDS**:

- **RDS** → tu instancia database-1
- Pestaña "**Connectivity & security**"
- Bajar a la sección "**Manage IAM roles**"
- En "**Add IAM role**" elegir rds-s3-import-role
- En "**Feature**" elegir **s3Import**
- "**Add role**"

Verificar la policy correcta

- **IAM** → **Roles** → buscar rds-s3-import-roleUTDT

Verificar que tiene la policy correcta.

Si el rol está bien creado, lo asociamos a la instancia RDS:

- **RDS** → tu instancia database-1
- Click en el nombre de la instancia
- Bajar y buscar **"Administración de IAM"** o **"IAM database authentication"**
- Buscar la sección **"Roles de IAM asociados"** o **"Manage IAM roles"**
- En **"Agregar rol"** elegir rds-s3-import-roleUTDT
- En **"Característica"** elegís **s3Import**
- **"Añadir función"**

Importamos el CSV desde S3

- **Paso 3:**

```
SELECT aws_s3.table_import_from_s3(  
    'personas',  
    'nombre,edad,ciudad',  
    '(format csv, header true)',  
    'clase-utdt',  
    'datos20260330.csv',  
    'sa-east-1'  
);
```

Resumen

Parte 1 — S3 básico:

- Crear un bucket
- Subir un archivo manualmente

Parte 2 — AWS CLI con S3:

- `aws s3 ls` — listar buckets
- `aws s3 cp` — subir y bajar archivos
- `aws s3 sync` — sincronizar carpetas
- `aws s3 rm` — borrar archivos

Parte 3 — S3 + Athena:

- Crear tabla externa apuntando a S3
- Consultar CSV con SQL sin base de datos

Parte 4 — S3 + RDS:

- Crear instancia PostgreSQL en RDS
- Importar datos desde S3 a PostgreSQL
- Consultar con SQL